Status: PREPARED DRAFT

Session 11B — Study Guide: MongoDB Connected Is Not the Same as MongoDB Used

Where the session ends

A real dependency-injection failure — left over from yesterday's genuinely interrupted class — is diagnosed and fixed. Every CRUD method on Projects moves from a private in-memory array to real, persisted MongoDB documents through Mongoose, verified by three independent real clients: `curl`, `mongosh`, and MongoDB Compass. The identical full pattern is then applied, independently, to Workspaces. By the end of class, both resources support create, read, update, and real filtering, all backed by a real database — proven to survive a server restart. The homework adds DELETE and completes CRUD on both resources.

The real starting point

Yesterday's class connected a real MongoDB database, built a real schema, and even wrote a service constructor that expected a real injected model — but was interrupted before registering that model with its own module, and before updating any of the service's own methods to actually use it. The result, this morning: the app won't even boot.

```
Nest can't resolve dependencies of the ProjectsService (?). Please make sure that the
argument "ProjectModel" at index [0] is available in the ProjectsModule context.
```

`"ProjectModel"` is not a name anyone typed — it's the literal token Nest derives from `Project.name`. Seeing this should suggest one thing: a registration step, for this exact model, in this exact module, never happened.

Module-scoped dependency injection

TS

```ts
MongooseModule.forRootAsync({
  inject: [ConfigService],
  useFactory: (config: ConfigService) => ({
    uri: config.get<string>('MONGODB_URI'),
  }),
}),
```

This opens ONE real connection. It does **not** make any model injectable anywhere. A separate call, registered in the OWNING feature module, does that:

TS

```
// ProjectsModule — the one line that was missing:
@Module({
  imports: [
    MongooseModule.forFeature([{ name: Project.name, schema: ProjectSchema }]),
  ],
  controllers: [ProjectsController],
  providers: [ProjectsService],
})
export class ProjectsModule {}
```

Nest's dependency injection is module-scoped: a provider can only receive what its OWN module has made available. `ProjectsService` already asked for `@InjectModel(Project.name)` — `forFeature` is the other half of that agreement.

The session's own central idea

```
MongoDB contains data
        ≠
API CRUD uses MongoDB
```

The moment the app boots for the first time (previous section), something surprising is already true: MongoDB now has real seed data, inserted live by `OnModuleInit`'s own hook the instant that boot succeeded. But `GET /projects` still returns the OLD in-memory array — completely unrelated to that collection. Two independent facts, both real, at the same time. The only way to really believe this is to check both yourself:

BASH

```
mongosh
use b4f-board
db.projects.find()
exit
curl -X GET localhost:3000/projects
```

`mongosh` shows the real collection. `curl` still shows the old array. They happen to look similar (both start from the same 3 seed rows) — which is exactly what makes this worth checking directly, not assuming.

Closing the gap for real

TS

```ts
async findAll(query: FindProjectsQueryDto): Promise<ProjectDocument[]> {
  const filter: Record<string, unknown> = {};

  if (query.name) {
    filter.name = { $regex: query.name, $options: 'i' };
  }

  if (query.key) {
    filter.key = query.key;
  }

  return this.model.find(filter).exec();
}
```

Every method becomes `async` because a database call is real I/O, not a synchronous array read — and, once the migration is complete, `findAll` reads real persisted documents AND applies the same exact-match-vs-substring-match filtering already designed earlier, run directly against that real data. Once every method is saved this way, the two facts above collapse into one:

```
API
 ↓
ProjectsService
 ↓
Mongoose Model
 ↓
MongoDB
```

Proving it, three ways

BASH

```bash
curl -X POST localhost:3000/projects -H 'Content-Type: application/json' -d '{"name":"Design System","key":"DESIGN"}'
mongosh --eval 'db.getSiblingDB("b4f-board").projects.find({ key: "DESIGN" })'
# Ctrl+C the dev server, then:
npm run start:dev
curl -X GET localhost:3000/projects
```

Create through the API; find it independently in `mongosh` — two unrelated clients agreeing. Then restart the SERVER (not MongoDB) and see the data survive — proof it was never living inside the Node process that just ended. MongoDB Compass, a GUI client, shows the exact same data a third way.

Common beginner mistakes

- **Treating a dependency-injection failure as a syntax error.** `npx tsc --noEmit` passes clean — this only shows up at runtime, in Nest's own boot error.
- **Assuming similar-looking data means the migration already worked.** Check with an independent client, don't assume from appearances.
- **Leaving `id: string` after adding `ParseIntPipe`.** The value is already a real `number`.
- **Forgetting `OnModuleInit`'s own guard.** Without `countDocuments() === 0`, every restart re-inserts and duplicates the seed data.
- **Assuming a direct database write is checked by Nest's own validation.** It isn't — that layer only runs for requests that go through the API itself.
- **Switching some queries to MongoDB's own `_id`.** This app keeps one consistent identifier (`id`) across every route and query.

Not yet

These exist in NestJS or MongoDB, but are deliberately **not** covered this session:

- Guards, Interceptors, custom decorators
- Swagger/OpenAPI
- Authentication, JWT, roles/permissions
- Database-level uniqueness constraints and indexes
- MongoDB's own `_id` as a public identifier
- Aggregation pipelines
- PostgreSQL, an ORM, Next.js

Glossary

- **Module-scoped dependency injection** — a provider can only receive what its own module (or an explicitly imported one) has made available; this is why `forFeature` must live in the module that actually uses the model.
- **Collection / document** — MongoDB's own equivalent of a table/row, with no fixed shape enforced by the database itself.
- **Schema** — the shape the APPLICATION declares and enforces for its own documents.
- **`mongosh`** — a command-line client connecting to a real, already-running MongoDB server.
- **MongoDB Compass** — MongoDB's own official GUI client, connecting the same way.
- **`OnModuleInit`** — a Nest lifecycle hook; `onModuleInit()` runs once, right after a module finishes wiring up.
- **`ParseIntPipe`** — a built-in pipe that converts a string to a number, or rejects it with a `400`.

Review questions

1. What exact registration was missing, and in which exact file, causing this morning's real boot failure?

2. Why doesn't "MongoDB already has the data" mean "the API is using it"?
3. What is the real difference between `MongooseModule.forRootAsync` and `MongooseModule.forFeature`?
4. Why does `OnModuleInit`'s seed logic check `countDocuments() === 0` first?
5. Why does restarting the Nest server no longer reset the data?

Answers

1. `MongooseModule.forFeature([{ name: Project.name, schema: ProjectSchema }])` was missing from `projects.module.ts` — `ProjectsService` already expected an injected model, but Nest had no model registered for that module to give it.
2. Because the API's own CRUD methods and the real database are two independent layers — a direct client (`mongosh`/Compass) sees the database regardless of what the application's own code is currently doing.
3. `forRootAsync` opens the ONE database connection for the whole app, built from `ConfigService`. `forFeature` registers a SPECIFIC model for injection inside a SPECIFIC feature module — a completely different, per-module registration.
4. Without it, every server restart would re-insert the same seed rows, duplicating them forever.
5. Data now lives in MongoDB, a separate, independently-running process (`mongod`) that persists to disk — restarting the Nest process never touches it, unlike every session before this one, where data lived inside the very process that just restarted.

What you should be able to explain now

- Why a connected database is not automatically a USED database — and how to check the difference yourself, with an independent client.
- Why Nest's dependency injection is module-scoped, and what `forFeature` actually registers.
- What actually changes, and what stays exactly the same, when a resource moves from an in-memory array to a real database.
- Why a direct MongoDB write never passes through Nest's own validation layer.